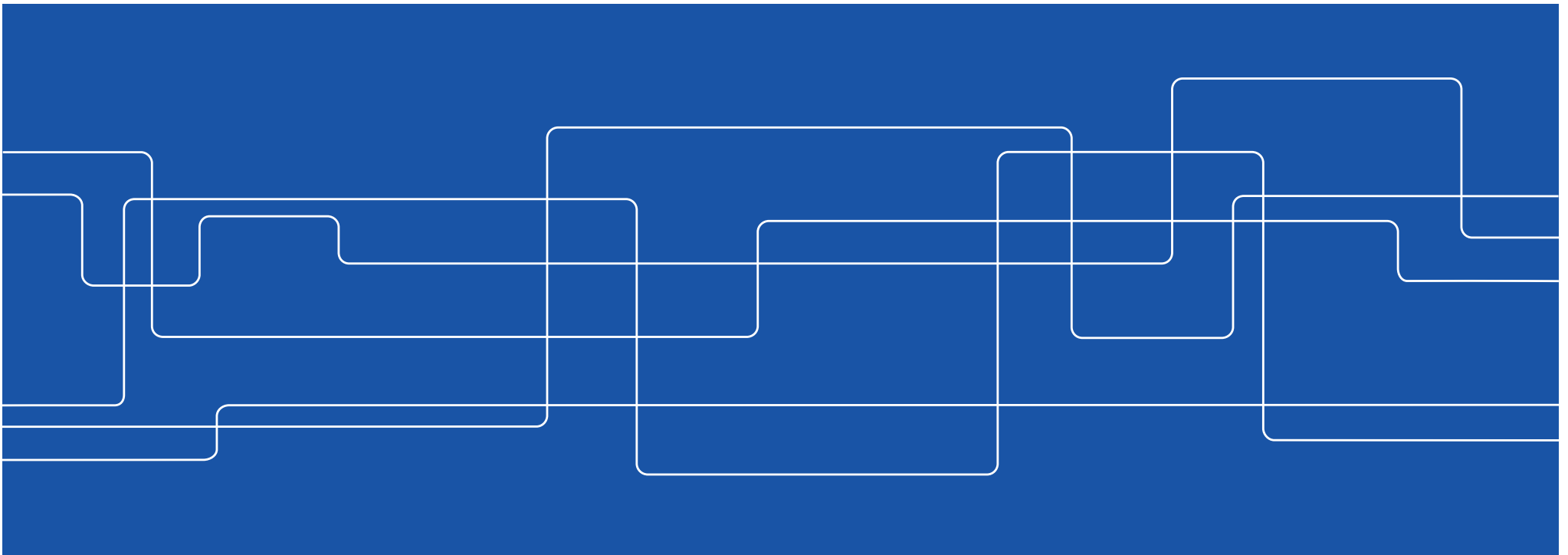




DD2380 Artificial Intelligence

Making decisions under uncertainty

Petter Ögren



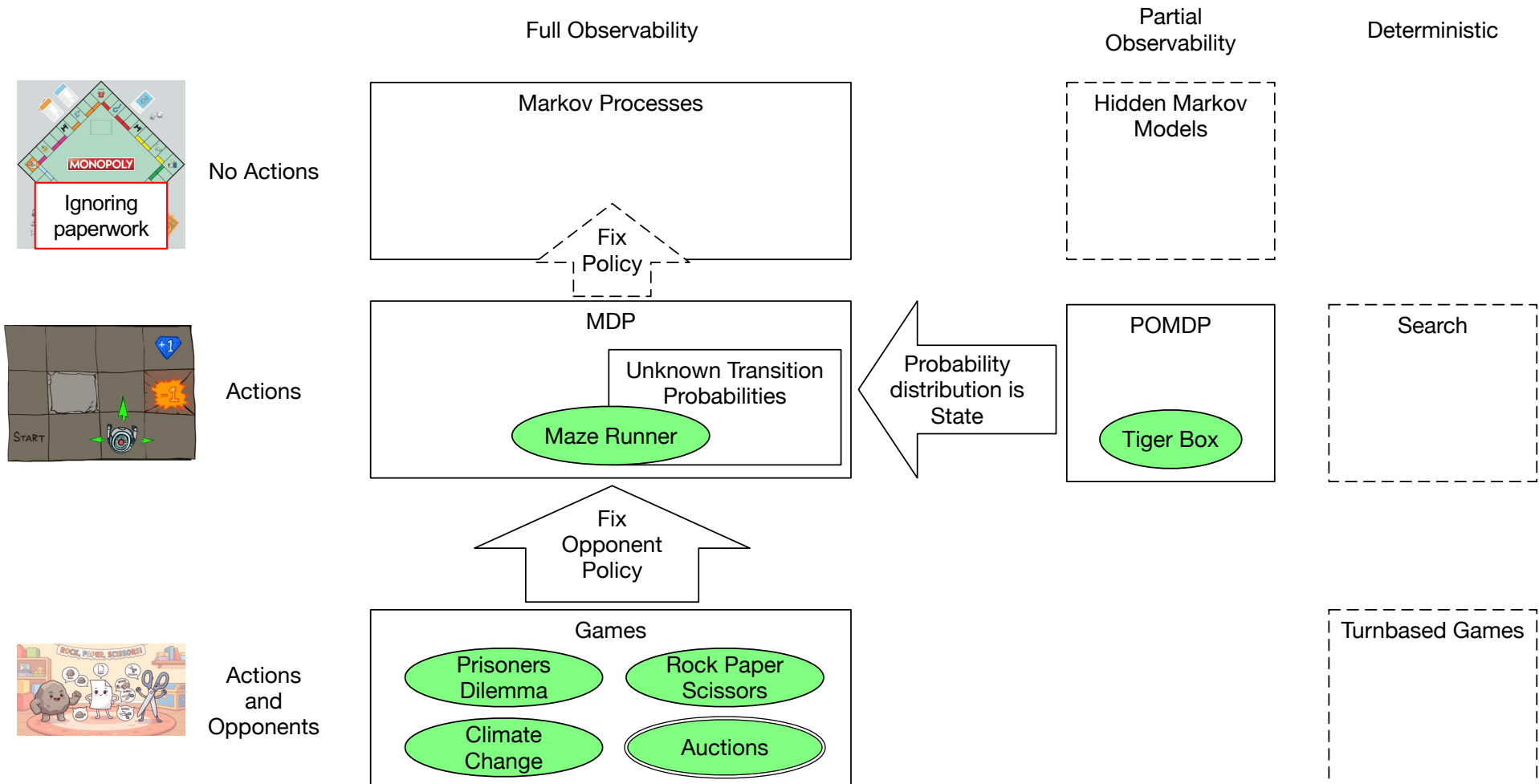


Additional reading

- Chapters 15, 16 and **17** in the book



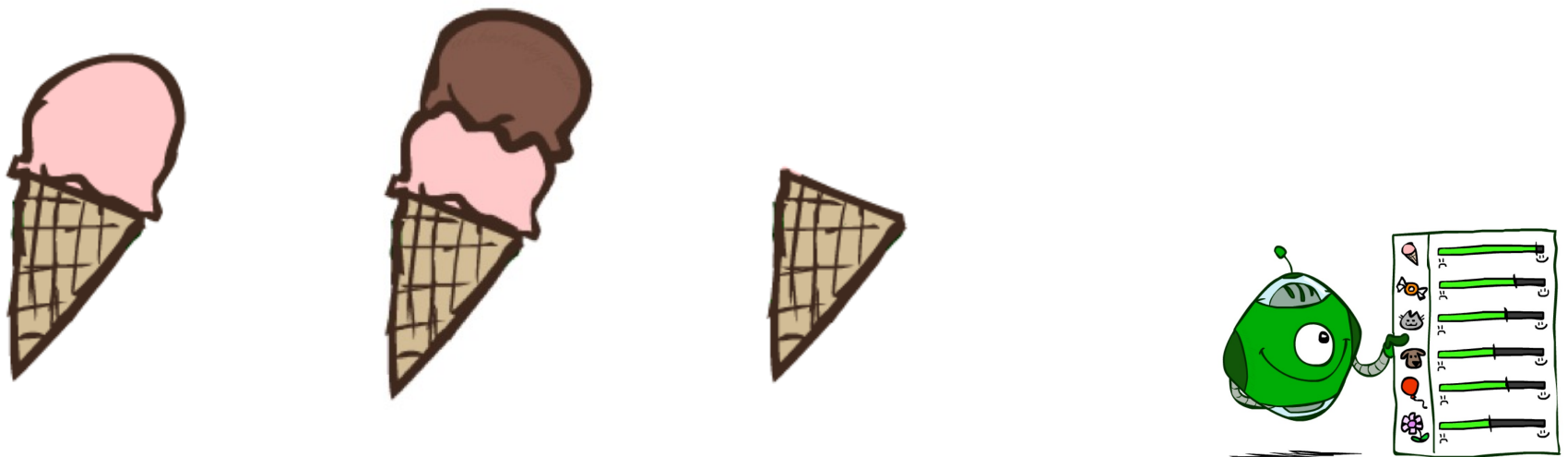
Problem Classes



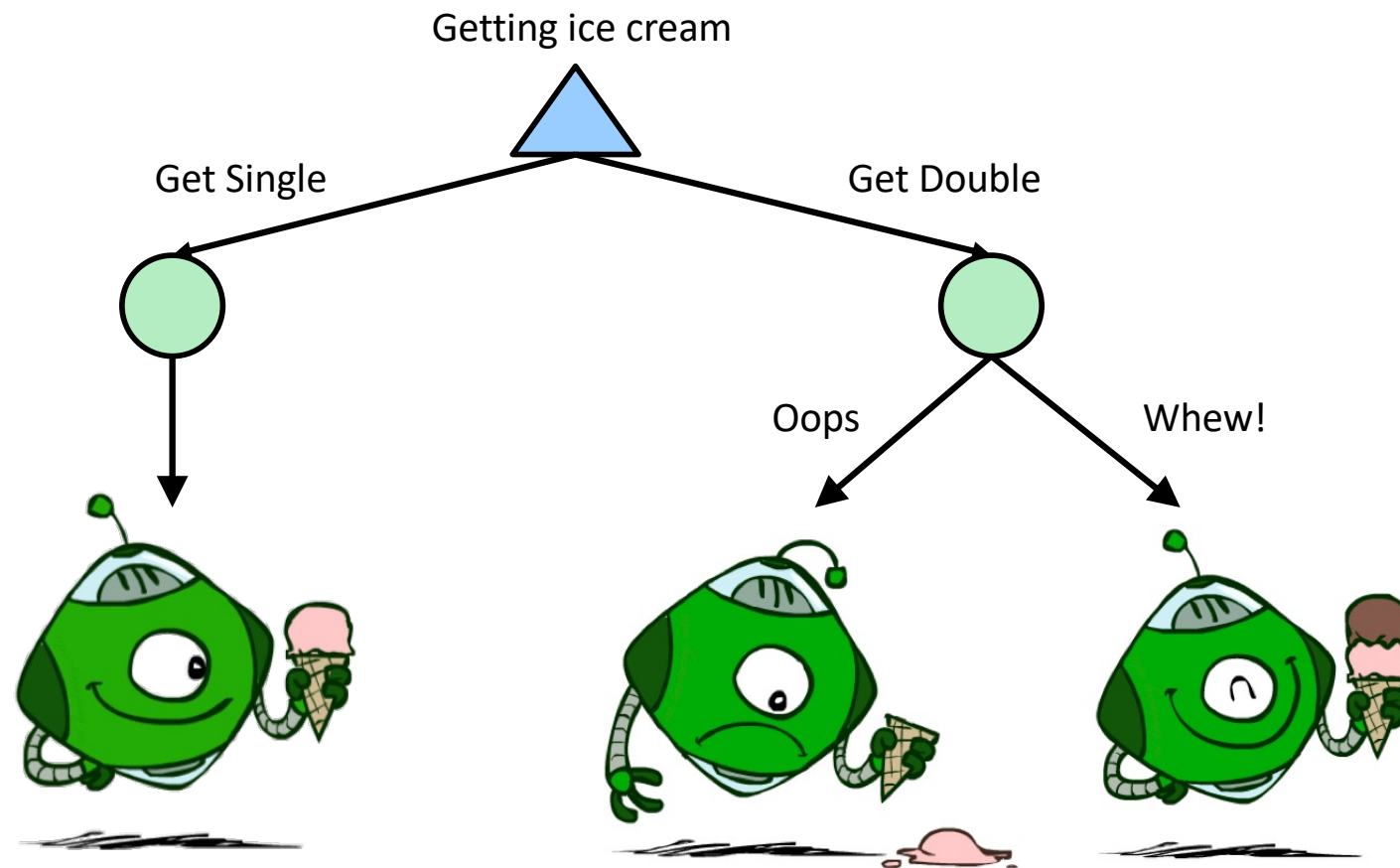


Utility function (to base decisions on)

- Captures an agent's preference between world states
 - See also module **Problem solving**
- Assigns single number to express desirability of a state
- The utility of state S is denoted by $U(S)$

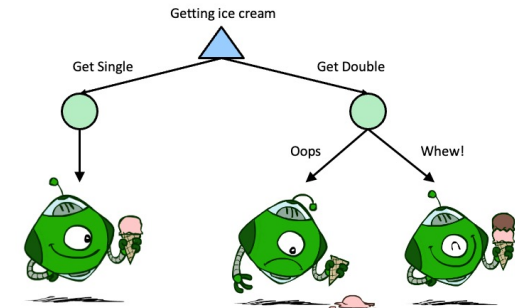


Utilities: Uncertain Outcomes





Maximum Expected Utility (MEU)



- Outcome of action is non-deterministic
- Result of action A is **Result_i(A)** (a state)
- Given evidence **E (measurements)** the probability for each result is

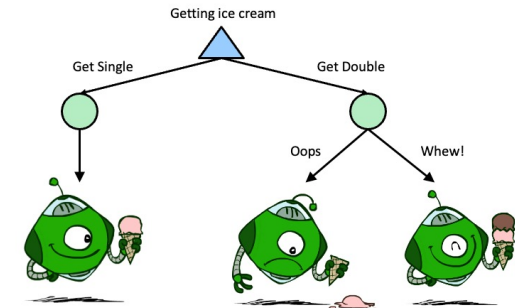
$$P(\text{Result}_i(A) \mid \text{Do}(A), E)$$

- Principle of maximum expected utility:
 - A rational agent picks an action that **maximizes the expected utility**, given its knowledge

$$E[U(A|E)] = \underset{A}{\operatorname{argmax}} \sum_i P(\text{Result}_i(A) \mid \text{Do}(A), E) U(\text{Result}_i(A))$$



MEU not so easy to apply



- State of the world? (partially observable at best)
- How to compute $P(\text{Result}_i(A)|\text{Do}(A),E)$? Requires a model of the world
- Also need to consider one action (one shot decision) vs. sequential decisions
- Utility of a state?

The value of information

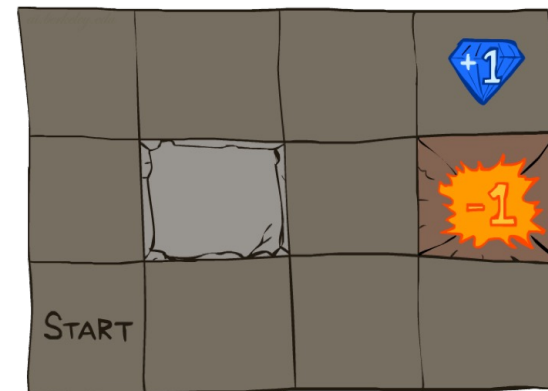
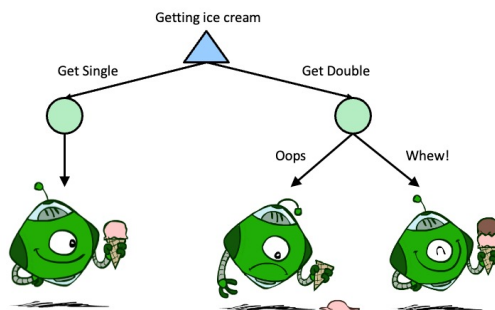
- Asking for information is one of the most important actions
 - Information is acquired through “sensing actions”
 - Information typically has a cost
 - Agent must ask itself, what information to ask for?
-
- Information has **value** if it might change your action



Sequential decisions

In sequential environments

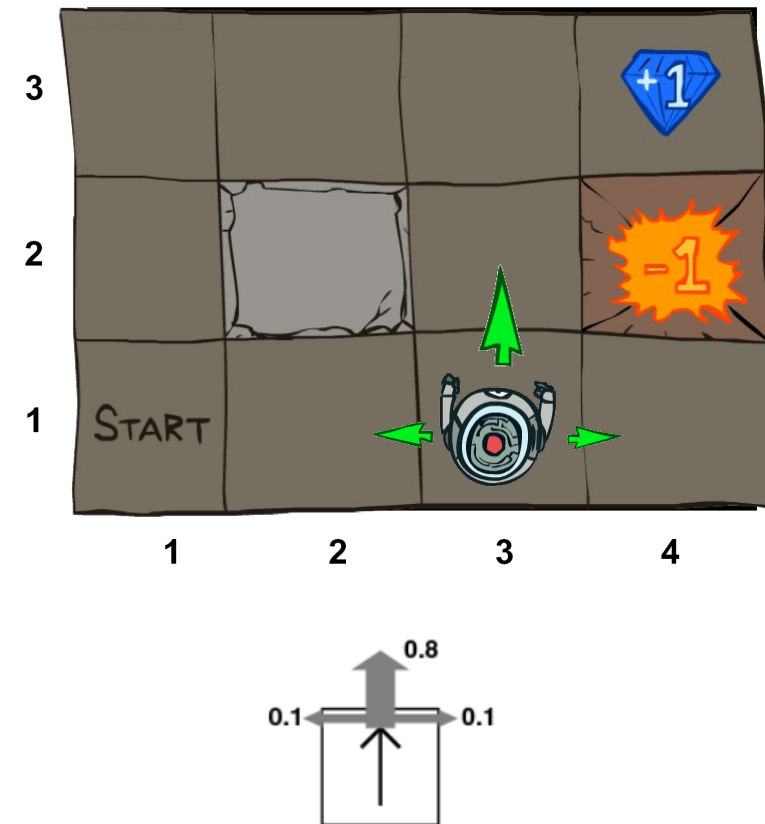
- A one step horizon (one action) often not good enough
- The agent's utility depends on a sequence of decisions



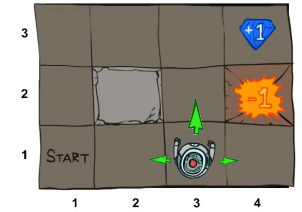


Seq. decisions example: Grid World

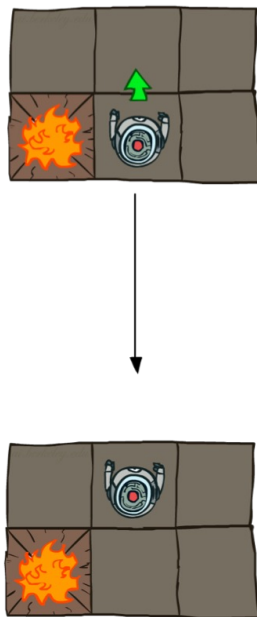
- A maze-like problem
 - The agent lives in a grid
 - Walls block the agent's path
- Noisy movement: actions (Up, Down, Left, Right) do not always go as planned
 - 80% of the time, the intended outcome occurs (if there is no wall there)
 - 20% of the time, the agent moves at right angles to the intended direction
 - If there is a wall in the direction the agent would have been taken, the agent stays put
- The agent receives rewards each time step
 - Small “cost of living” reward each step (often negative)
 - Big rewards come at the end (good or bad)
- Goal: maximize sum of rewards



Grid World Actions

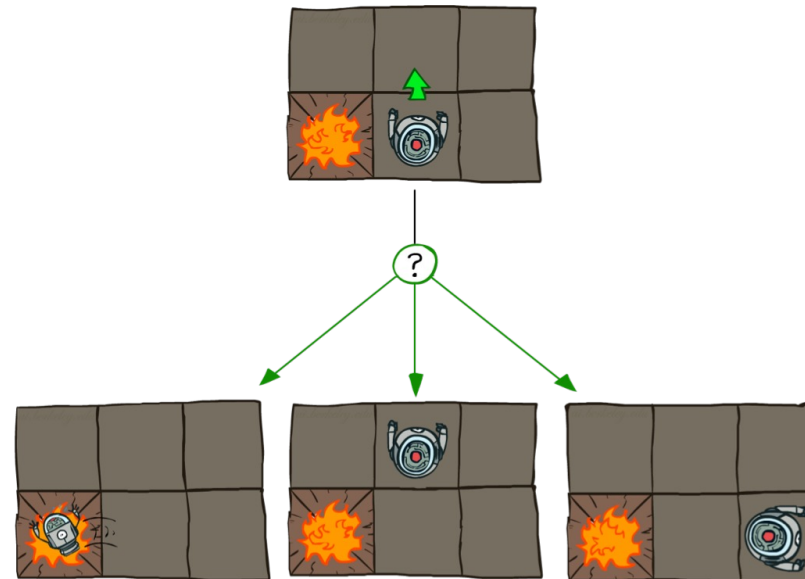


Deterministic Grid World



Plan: Up, Up, Right, Right, Right

Stochastic Grid World



Cannot create a plan ahead of time!

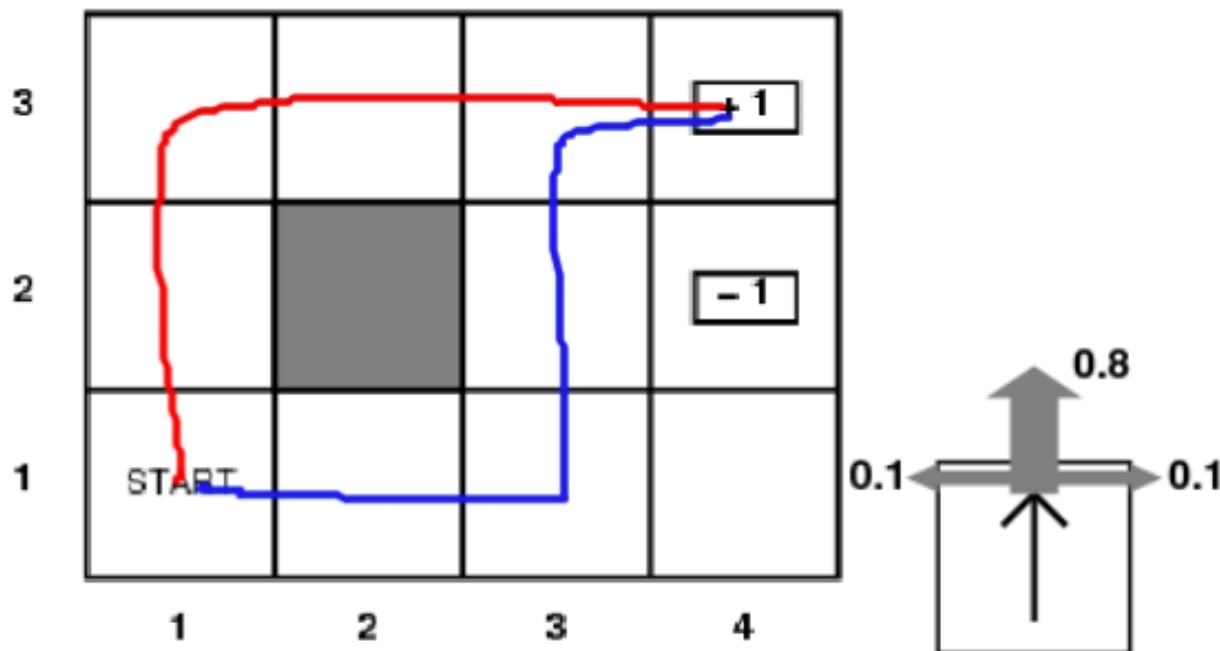


What is the probability that a predefined plan succeeds?

Plan: Up, Up, Right, Right, Right

Probability to succeed: 0.32776

$$0.8^5 + (0.1^4 \cdot 0.8) = 0.32776$$





Connection to past lectures?

- Remember Markov Models
 - Each square is a state in the previous example
- The transition function defines the probability transitioning from one state to another.
- In decision making, **each action is associated with its own transition function**
 - select action to maximize the chance of achieving some goal

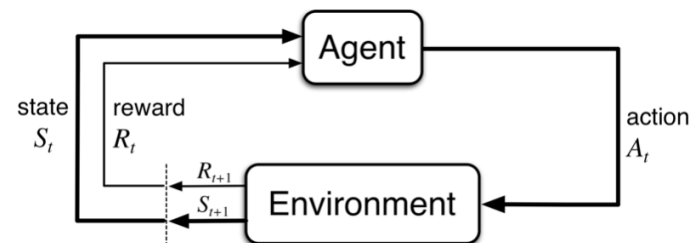
“Markov” generally means that given the present state, the future and the past are independent



Markov Decision Process (MDP) Formulation

Mathematical model: Markov Decision Process (S, A, T, R, γ)

- State space S
- Action space A
- Environment Transition model T
- Reward function R
- Discount factor γ
- A start state S_0
- Maybe a terminal state



$$T(s, a, s') : p(s_{t+1} = s' | s_t = s, a_t = a)$$

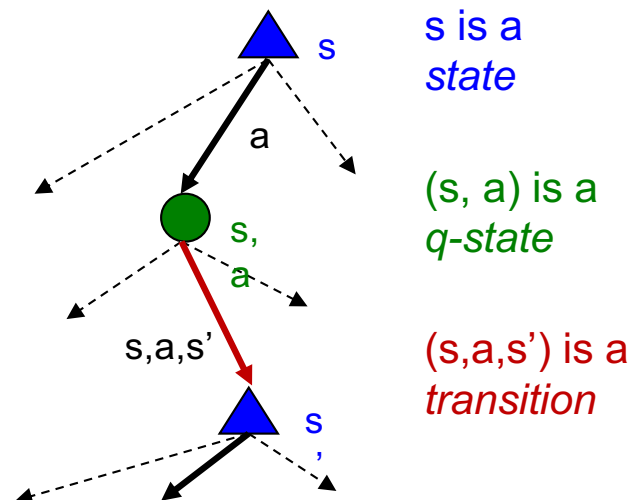
$R(s, a, s')$: immediate reward for transitioning from state s to state s' due to action a

Frequently reward depends only on the state, so we usually write $R(s)$



Transition model

- Now function of the action
- $T(s,a,s')$ (first order Markov assumption*)
Probability to reach s' starting from s given action a .

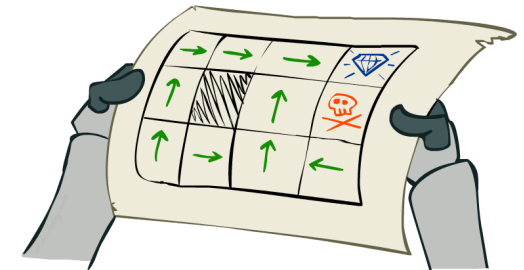




Solution to MDP

In deterministic single-agent search problems, we wanted an optimal **plan**, or sequence of actions, from start to a goal

A solution to an MDP cannot be a fixed plan (non-deterministic world, need to sense state)



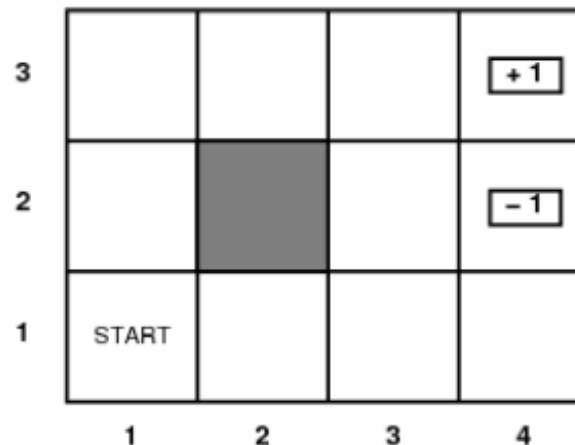
For MDPs, we want an optimal **policy** $\pi^*: S \rightarrow A$

- A policy π gives an action for any state the agent might reach
- An optimal policy is one that maximizes expected utility if followed



Ex: Optimal policy π^*

- An example from the book
- Assume agent gets reward $R(s)$ for being in state s
 - $R([4,3]) = +1$ (Go to goal)
 - $R([4,2]) = -1$ (Avoid trap)
 - $R(\text{rest}) = -0.04$ (Get to goal quickly, “living cost”)

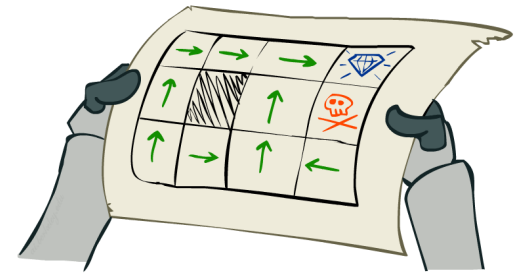




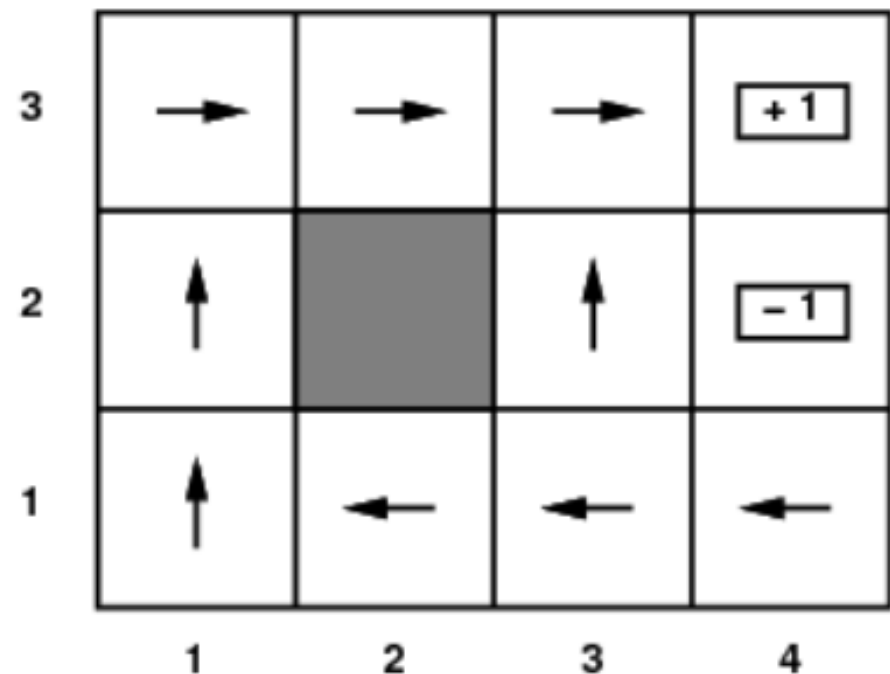
Ex: Optimal policy π^*

Optimal policy for the previous problem
Tells us what to do in each state

Actual path only known afterwards
because **actions**
are non-deterministic



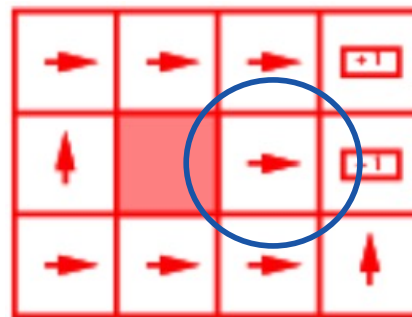
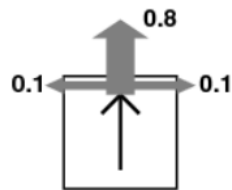
Which policy is
the optimal one?



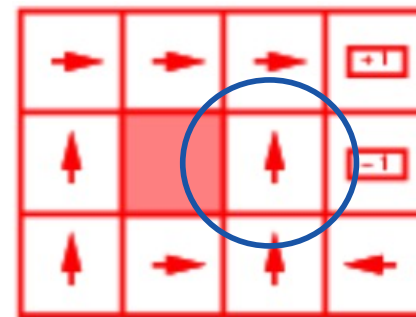


Optimal policy depends on R and T

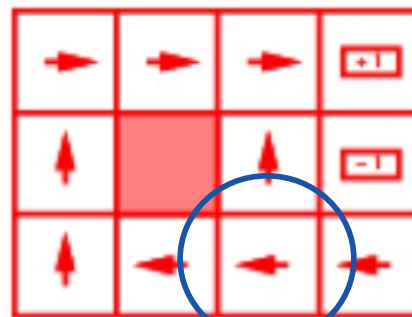
Different behaviors for different “living cost” Reward values



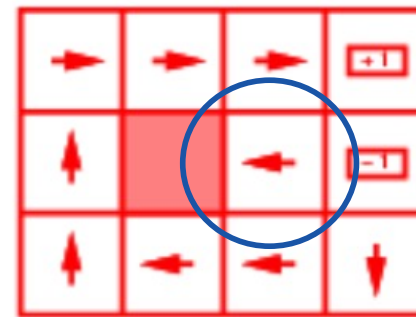
$r = [-\infty : -1.6284]$



$r = [-0.4278 : -0.0850]$



$r = [-0.0480 : -0.0274]$



$r = [-0.0218 : 0.0000]$

r : “cost of living”



How good is a policy?

How to measure the quality of a policy?

→ Measure expected utility over the history
(stochastic Env means that we need to use expectations)

Optimal policy, π^* : highest possible expected utility



Utility of sequences

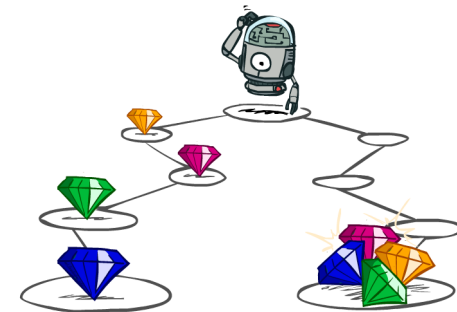
Is this good?

$$U_h([s_0, s_1, \dots, s_n]) = R(s_0) + R(s_1) + \dots + R(s_n)$$

What preferences should an agent have over reward sequences?

More or less? [1, 2, 2] or [2, 3, 4]

Now or later? [1, 1, 1, 1] or [0, 0, 0, 4]





Utility of sequences: Additive rewards

Additive rewards:

$$U_h([s_0, s_1, \dots, s_n]) = R(s_0) + R(s_1) + \dots + R(s_n)$$

What about infinite sequences?

- Might get ∞ without terminal state.
- How to compare ∞ and ∞ ?

$[1, 1, 1, 1, \dots]$ or $[2, 2, 2, 2, \dots]$?



Utility of sequences: Discounted rewards



1

Worth Now

- Idea: Give less weight to future rewards (e.g. rewards decay exponentially)
- Captures that rewards “tomorrow” is less certain
- Use discount factor γ , $0 < \gamma < 1$

$$U_h([s_0, s_1, \dots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots$$

We will use this for
evaluating policies

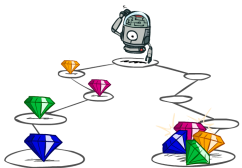


γ

Worth Next Step

- Gives bounded utility

$$R(s) \leq R_{max} \Rightarrow U_h \leq \sum_{i=0}^{\infty} \gamma^i R_{max} = R_{max} \frac{1}{1-\gamma}$$



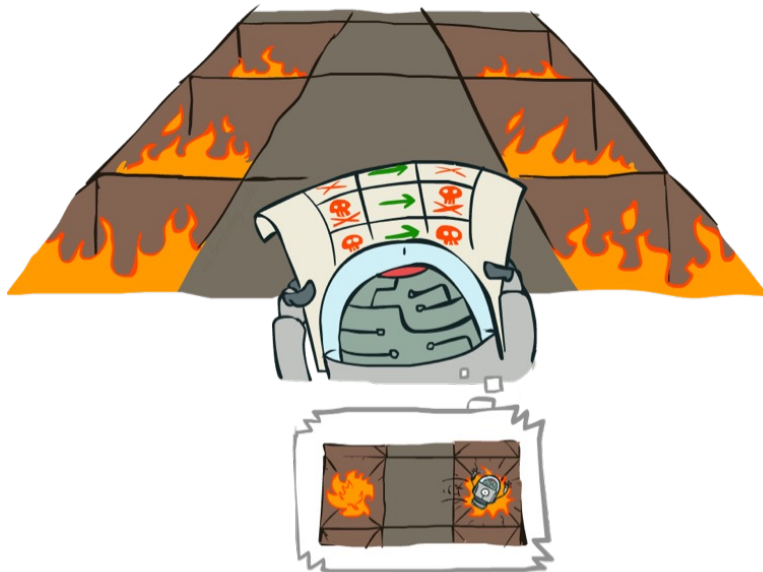
γ^2

Worth In Two Steps

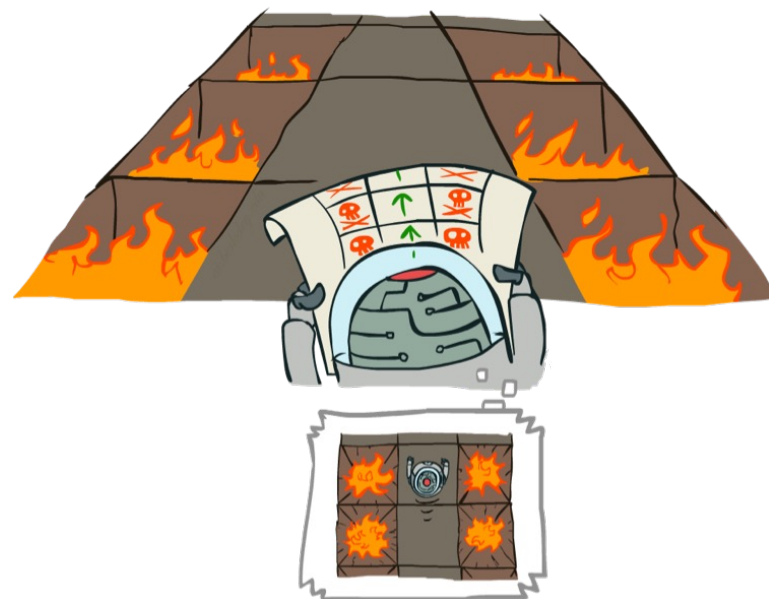
Example: Policy Evaluation

$$U^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{k+1} \middle| S_0 = s \right]$$

Always Go Right



Always Go Forward



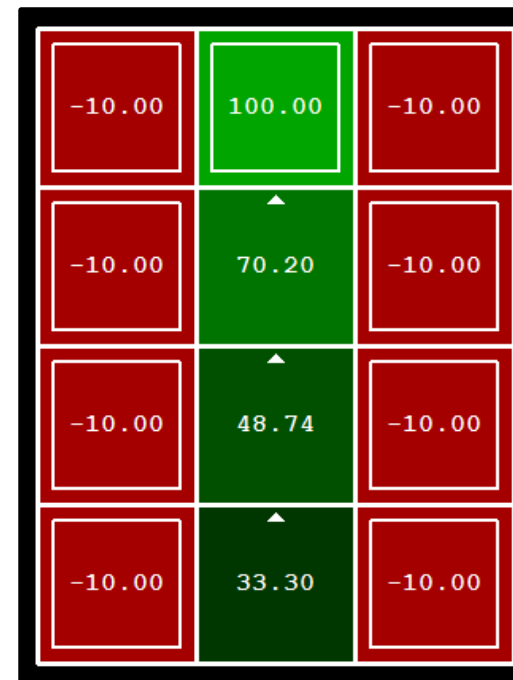
Example: Policy Evaluation

$$U^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{k+1} \mid S_0 = s \right]$$

Always Go Right



Always Go Forward





Value of optimal policy $U(s)$

$$U^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{k+1} \mid S_0 = s \right]$$

$$U(s) = \max_{\pi} U^\pi(s) = \max_{\pi} \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{k+1} \mid S_0 = s \right]$$

Note:

* Is sometimes used to denote *optimal*

U , V is same thing

$$U^*(s) = U(s) = V^*(s) = V(s)$$



Value of optimal policy $U(s)$

$$\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{k+1} \mid S_0 = s \right]$$

Can we re-create the optimal policy from U ?

A* does this

$U(s)$ deterministic

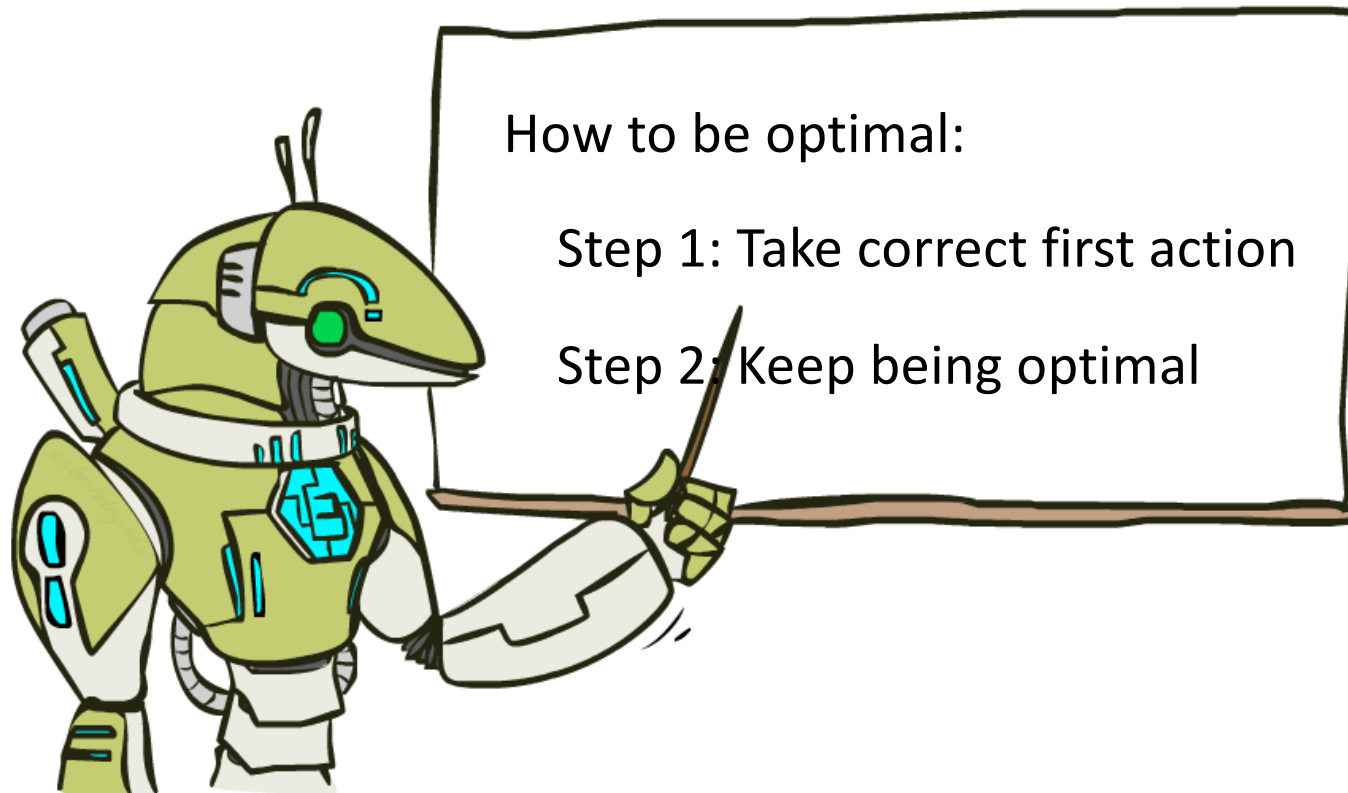
0.80	0.90
0.70	
0.60	0.70
OPTIMAL VALUES	

A bit similar to A*

$U(s)$ probabilistic

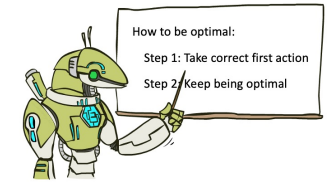
0.67	0.81	0.94	1.00
0.54		0.62	-1.00
0.41	0.32	0.45	0.19
OPTIMAL VALUES (VALUE ITERATION)			

Bellman Equation $\leftarrow$ how to find $U(s)$?





How to find $U(s)$?

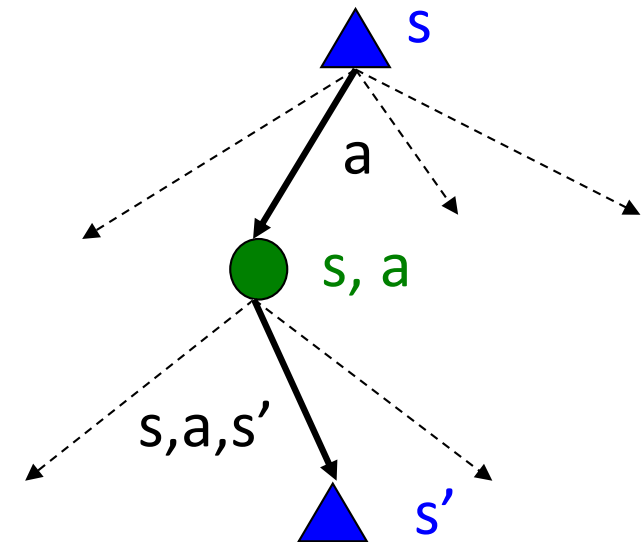


$U(s)$ is the solution to the Bellman equation

$$U(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s') \right)$$

Iteration can solve equations...

$$U_{i+1}(s) \leftarrow \max_a \left(R(s, a) + \gamma \sum_{s'} T(s, a, s') U_i(s') \right)$$



“Value Iteration”
algorithm

Converges to unique optimal solution

Stop iterations when largest change in utility for any state is small enough

Can show that

$$\|U_{i+1} - U_i\| < \epsilon \frac{1 - \gamma}{\gamma} \Rightarrow \|U_{i+1} - U\| < \epsilon$$



- $U(s)$: Utility of following policy
- $Q(s,a)$: Utility of *first doing a*, then following *policy*

$$U(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s') \right)$$

$$a(s) = \operatorname{argmax}_a \left(R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s') \right)$$

$$U(s) = \max_a Q(s, a)$$

$$a(s) = \operatorname{argmax}_a Q(s, a)$$

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s')$$

0.67	0.81	0.94	1.00
0.54		0.62	-1.00
0.41	0.32	0.45	0.19

OPTIMAL VALUES (VALUE ITERATION)

0.58	0.71	0.84	1.00
0.56	0.67	0.60	0.81
0.48	0.71	0.59	0.94
0.54		0.62	-1.00
0.44	0.44	0.53	-0.68
0.34		0.23	
0.41	0.24	0.45	-0.76
0.32	0.25	0.29	0.32
0.30	0.24	0.31	0.16
			0.19
			-0.02
			0.11

OPTIMAL Q-VALUES (VALUE ITERATION)



Value Iteration Example

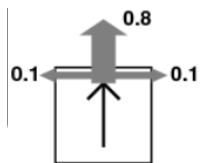
3				+1
2				-1
1	START			
	1	2	3	4

Values after 1 iteration

3	0	0	0	+1
2	0		0	-1
1	START	0	0	0
	1	2	3	4

Values after 2 iterations

3			?	+1
2				-1
1	START			
	1	2	3	4



$\gamma = 0.9$,
living reward = 0

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s')$$

$$Q((3,3), \text{right}) = 0 + 0.9 * (0.8*1 + 0.1*0 + 0.1*0) = 0.72$$

$$Q((3,3), \text{left}) = \dots,$$

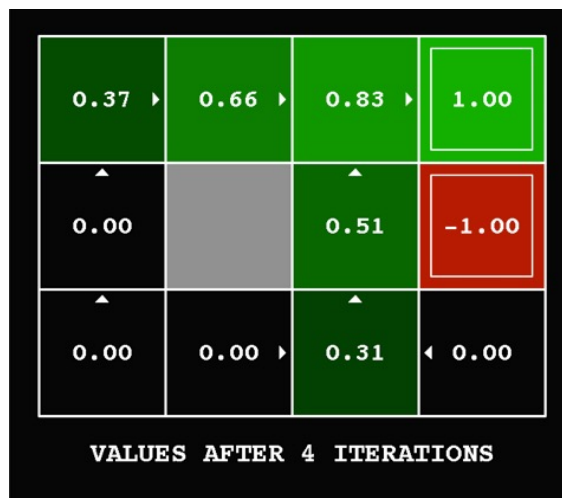
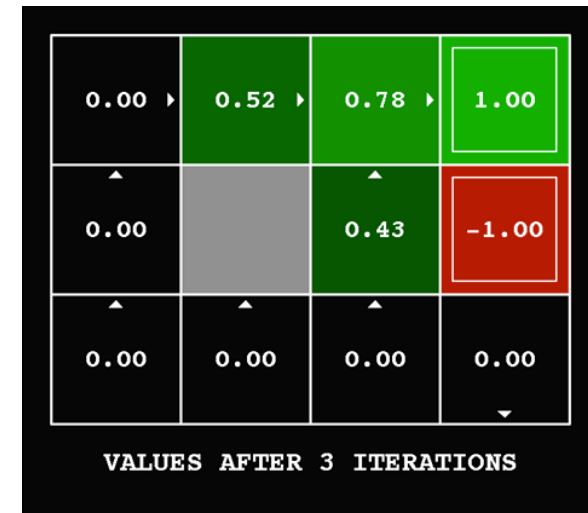
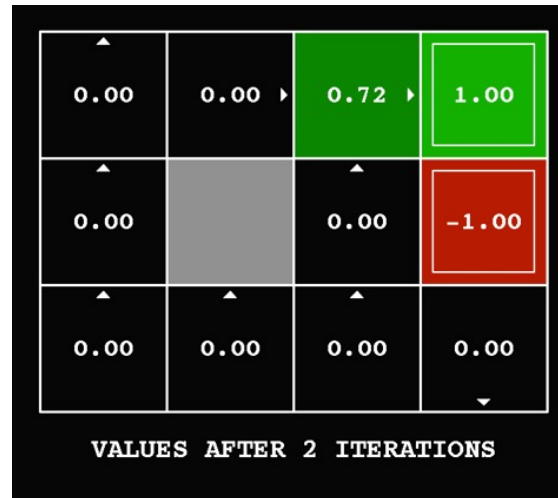
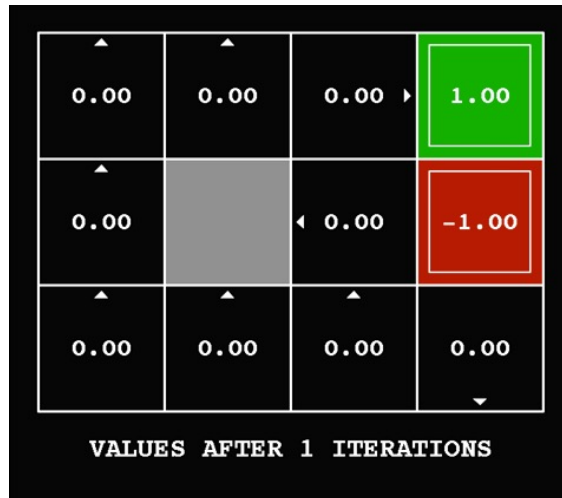
$$Q((3,3), \text{up}) = \dots,$$

$$Q((3,3), \text{down}) = \dots$$

$$U(3,3) := \max_a Q((3,3), a) = 0.72$$



Value Iteration Example





Value Iteration Example

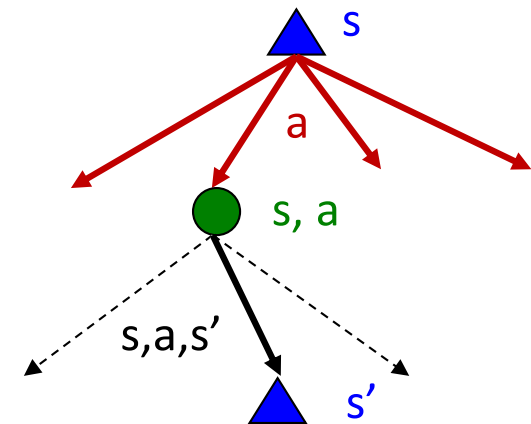




Problems with Value Iteration

Value iteration repeats the Bellman updates:

$$U_{i+1}(s) \leftarrow \max_a \left(R(s, a) + \gamma \sum_{s'} T(s, a, s') U_i(s') \right)$$



Problem 1: It's slow – $O(S^2A)$ per iteration

Problem 2: The “max” at each state rarely changes

Problem 3: The policy often converges long before the values

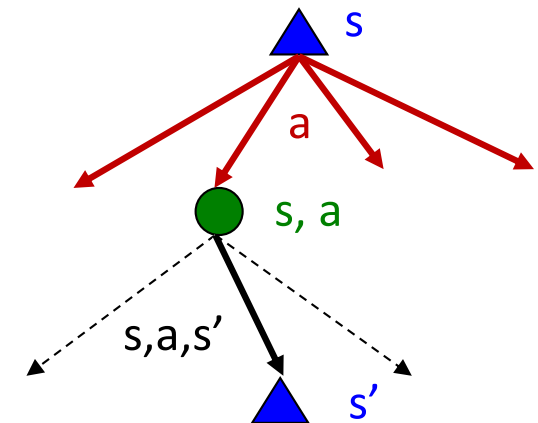


Utility of: Given policy vs Optimal policy

$$U(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s') \right)$$

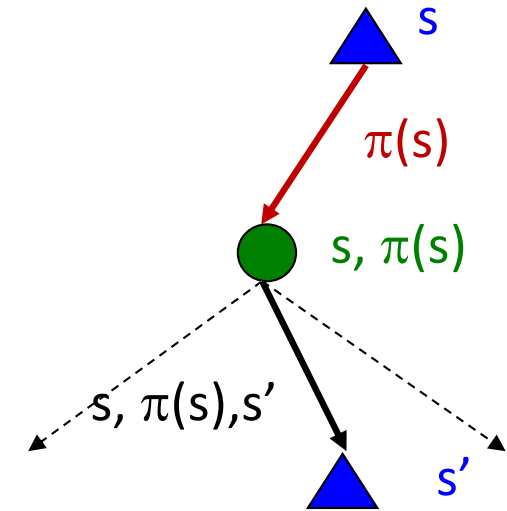
$$U^\pi(s) = \left(R(s, \pi(s)) + \gamma \sum_{s'} T(s, \pi(s), s') U(s') \right)$$

Given policy $\pi(s)$





Algorithm: Policy iteration



Choose an arbitrary policy

Loop until policy does not change any more

1. Policy evaluation: Compute the value function $U(s)$ until convergence, given the fixed policy (not optimal values):

$$U^\pi(s) = \left(R(s, \pi(s)) + \gamma \sum_{s'} T(s, \pi(s), s') U^\pi(s') \right)$$

2. Policy improvement: Given this value function, improve the policy for each state (choose action leading to a better state value)

$$a(s) = \operatorname{argmax}_a \left(R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s') \right)$$



Value Iteration vs. Policy Iteration

Both value iteration and policy iteration compute the same thing (optimal policy)
Both are dynamic programs for solving MDPs

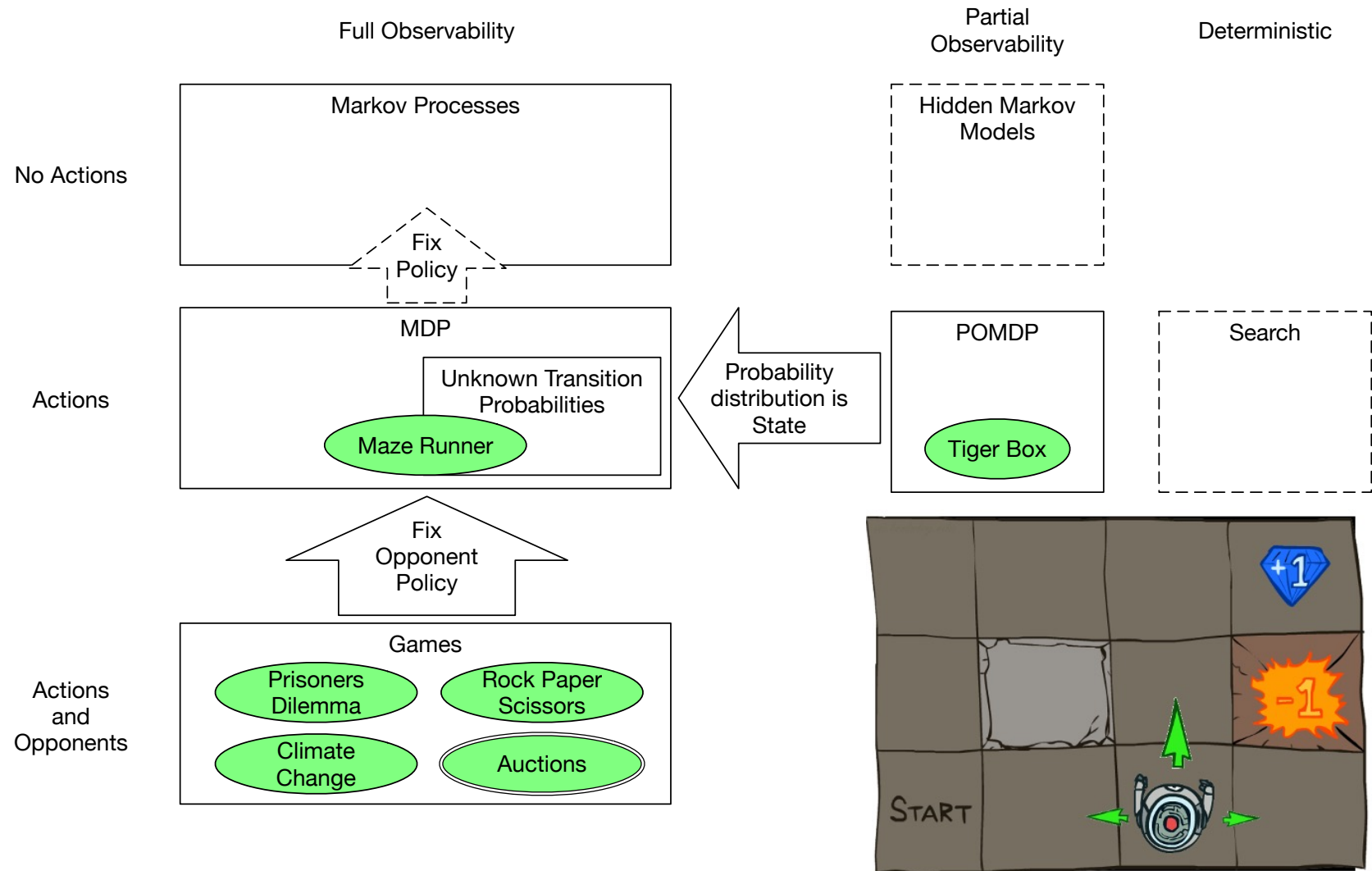
In **value iteration**:

- Every iteration updates both the values and (implicitly) the policy
- We don't track the policy, but taking the max over actions implicitly recomputes it

In **policy iteration**:

- We do several passes that update utilities with fixed policy (each pass is fast because we consider only one action, not all of them)
- After the policy is evaluated, a new policy is chosen (slow like a value iteration pass)
- The new policy will be better (or we're done)
- Can converge (much) faster under some conditions (large nr of actions but where max action doesn't change much)

Problem Classes





Partially observable environment

What if the environment is not observable?

i.e., the state that the agent is in is also uncertain

Remember: there are no sensors that let us see everything, so this is the normal case!





Partially observable environment

- What if the environment is not observable?
- Cannot execute policy $\pi(s)$ since **the state is unknown!**
- Results in a Partially Observable MDP aka **POMDP**
- Sensors provide observations of the environment
- Observation model $O(s,o)$ gives probability of making observation o in state s

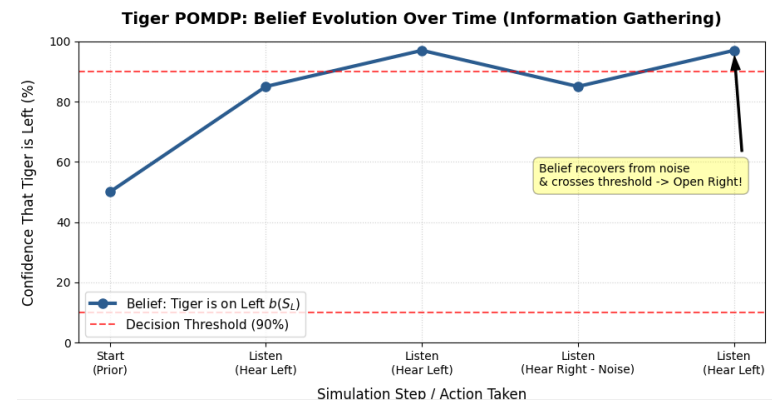
The Tiger POMDP





The Tiger POMDP

- States
 - SL (tiger is left)
 - SR (tiger is right)
- Actions
 - Open left
 - Open right
 - Listen
- Observations
 - 85% accurate 15% false
- Reward
 - Gold +10
 - Tiger -100
 - Listen -1



Belief state: $P(\text{tiger is left})$

Trick: Use belief state as state
POMDP $\rightarrow$ MDP (can solve!)

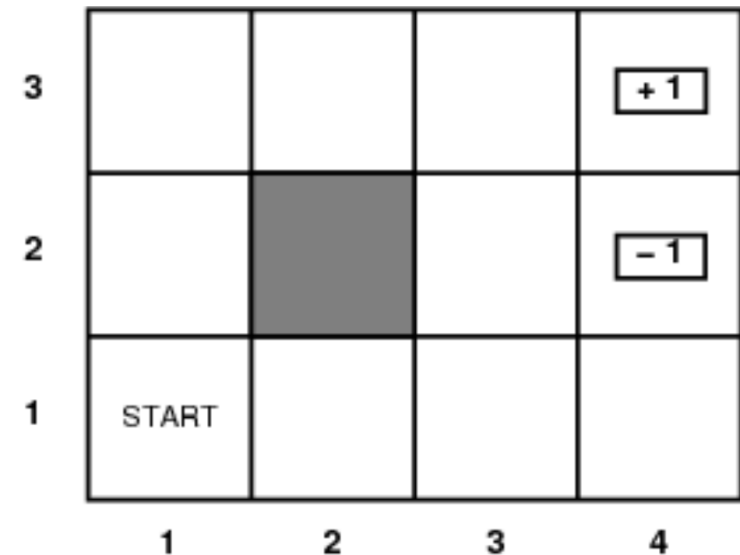


Belief state

- Idea: Use the belief state instead of the actual state
- Definition: Belief state b is a probability distribution over possible states
- $b(s)$ is the probability of being in state s

- Example: Initial belief state assuming that the agent can be anywhere except +1 or -1?

$$\langle \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, 0, 0 \rangle$$





Belief state update

- Need to update the belief state as we go along
- Assume **belief state b** and **action a**
 - our “best guess” about current situation
- Update?
 - $b'(s') = \text{some function of } b(s), a, s \text{ and } o \dots$



Belief state update

- Assume belief state b , action a and new observation o
- Update:

$$b'(s') = \alpha O(s', o) \sum_s T(s, a, s') b(s)$$

α is a normalization factor such that $\sum_{s'} b'(s') = 1$

Bayes rule with:

- $b(s) = p(s)$ ("prior")
- $b'(s') = p(s'|s, a, o)$ ("posterior")
- $\text{Sum}[T(s, a, s') b(s)] = p(s'|a, s)$ ("prediction")
- **$O(s', o) = p(o|s')$** = $p(o|s', s, a)$ { o indep of s and a , given s' }



Solving POMDP

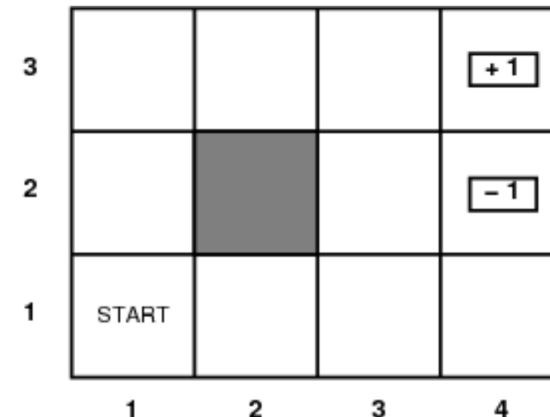
Key insight:

Optimal action depends on belief state and not actual state!

Optimal policy $\pi^*(b)$

Decision cycle:

1. Execute action $a = \pi^*(b)$
2. Receive observation
3. Update belief state





Turn a POMDP into an MDP

Introduce

- $\tau(b,a,b')$ – probability of reaching belief state b' from b given action a
- $\rho(b) = \sum b(s)R(s)$ – expected reward for the belief state

$\tau(b,a,b')$ and $\rho(b)$ define an observable MDP

Optimal MDP policy $\pi^*(b)$ is also optimal for the original POMDP

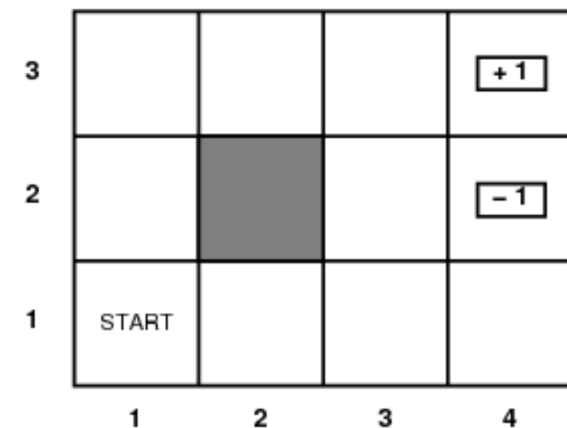
More on Chapter 17.4 of the book



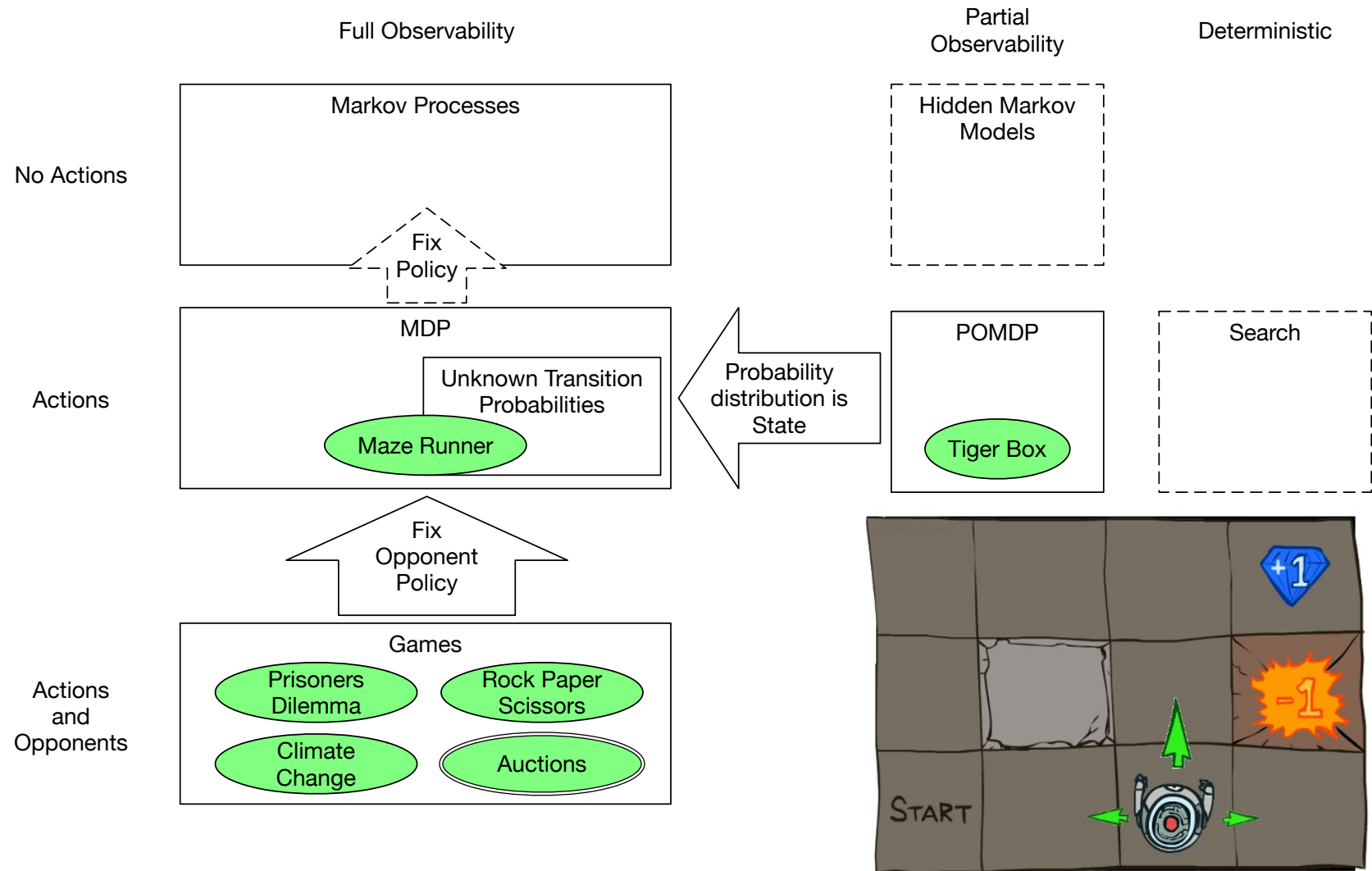
So is it simple?

- Sounds “simple” at first, BUT...
- POMDPs remain hard to solve exactly, even with advances in algorithms and computing power
- Belief state is a probability distribution!
- Compare MDP and POMDP in the 4x3 world
 - MDP state : The position of the agent, i.e. 1 discrete variable with 11 possible values.
 - POMDP belief state: 11 dimensional vector of continuous variables!!!

$$\langle \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, 0, 0 \rangle$$



Problem Classes





Game theory

- **Other agents also introduce uncertainty** (what decision do they make?)
- Game theory studies settings where multiple parties (agents) each have
 - different preferences (utility functions)
 - different actions that they can take
- Each agent's **utility** (potentially) depends on all agents' actions
 - What is optimal for one agent depends on what other agents do
- Agents can rationally form **beliefs** over what other agents will do, and (hence) how agents should **act**
 - Important area not only to make money at gambling
 - Useful for acting and predicting behavior of others



Game theory

Two main areas:

- Agent design:

What is the best strategy for the individual, assuming others act optimally?

- Mechanism design:

“Given that agents pick rational strategies, what game should we design?” i.e., How do we construct rules such that the best policy for the individual agents are also for the good of all? (e.g. multi-agent distributed systems)



Agent Design: Single Move Games

Main components:

- Players or agents
- Actions
- Payoff-matrix: Gives utility for each player for each combination of actions

Strategy = policy



Example: Prisoner's dilemma

Burglars Alice and Bob are caught
Interrogated separately by the the police



Rules of the game:

- Both testify: 5 years in prison each
- Both refuse to confess: 1 year each
- If only one testifies: One person 10 years, other goes free

What to do?



Prisoner's dilemma

Each agent wants to maximize expected utility

Payoff matrix

	Alice: testify	Alice: refuse
Bob: testify	A=-5, B=-5	A=-10, B=0
Bob: refuse	A=0, B=-10	A=-1, B=-1

Rules of the game:

Both testify: 5 years in prison each

Both refuse to confess: 1 year each

If only one testifies: One person 10 years, other goes free



Prisoner's dilemma cont'd

Alice analysis:

- Bob testifies: Alice's best action is to **testify** ($-5 > -10$)
- Bob refuses: Alice's best action is to **testify** ($0 > -1$)



	Alice: testify	Alice: refuse
Bob: testify	A=-5, B=-5	A=-10, B=0
Bob: refuse	A=0, B=-10	A=-1, B=-1

- That is, best to testify! – dominant strategy (highest payoff regardless of other agent's action)
- Bob feels the same way.



Nash equilibrium

Nash equilibrium:

- Set of strategies such that no player can benefit by changing their strategy while the other players keep their strategies unchanged

Alice and Bob are stuck in a Nash equilibrium

	Alice: testify	Alice: refuse
Bob: testify	A=-5, B=-5	A=-10, B=0
Bob: refuse	A=0, B=-10	A=-1, B=-1

Nash equilibrium in Climate Change



Nash equilibrium:

- Set of strategies such that no player can benefit by changing their strategy while the other players keep their strategies unchanged

We are all stuck in a Nash equilibrium

	Alice: Diesel	Alice: Electric
Everyone else: Diesel	$A = -5e99 + 10$	$A = -5e99$
Everyone else: Electric	$A = 100 + 10$	$A = 100$

Even worse:
N-player scale
Asymmetric impact
Temporal lag

Diesel subsidies



Game theory

Two main areas:

- Agent design:

What is the best strategy for the individual, assuming others act optimally?

- Mechanism design:

“Given that agents pick rational strategies, what game should we design?” i.e., How do we construct rules such that the best policy for the individual agents are also for the good of all? (e.g. multi-agent distributed systems)



Mechanism design example: Auctions

- Single good
- Each player has utility value v_i for the good
- Utility value known only to the bidder
- Bidders make bids b_i , highest bid wins

Example: English auction (ascending bid)

- Bids are incremented
 - Continues until only one bidder
 - Bidder with the highest v_i wins
- 
- A cartoon illustration of an English auction. An auctioneer stands at a podium, holding a gavel and a sign that says 'SOLD!'. He is shouting 'Going once, going twice... SOLD!'. A large crowd of bidders is seated in front of him, many holding up signs with numbers. A digital display on the right shows 'Let \$22,000'. The scene is set in a grand, ornate hall with high ceilings and chandeliers.
- Strategy: bid as long as price below your value
 - Result:
 - Pays $b_m + d$ (b_m highest among other, d =increment)
 - Requires a lot of communication



Example: Sealed bid auction

- Each bidder makes a single bid
- Highest bid wins
- Strategy: bid $\min(v_i, b_m + \varepsilon)$
- (b_m believed max of other bidders)
- Player with highest v_i might not get the good
- Requires less communication but more “work”
(need to estimate b_m)





Example: Vickrey auction

- Same as sealed-bid auction but winner pays second highest bid
- Strategy: bid your own value v_i
- Simple and minimal communication



Common applications

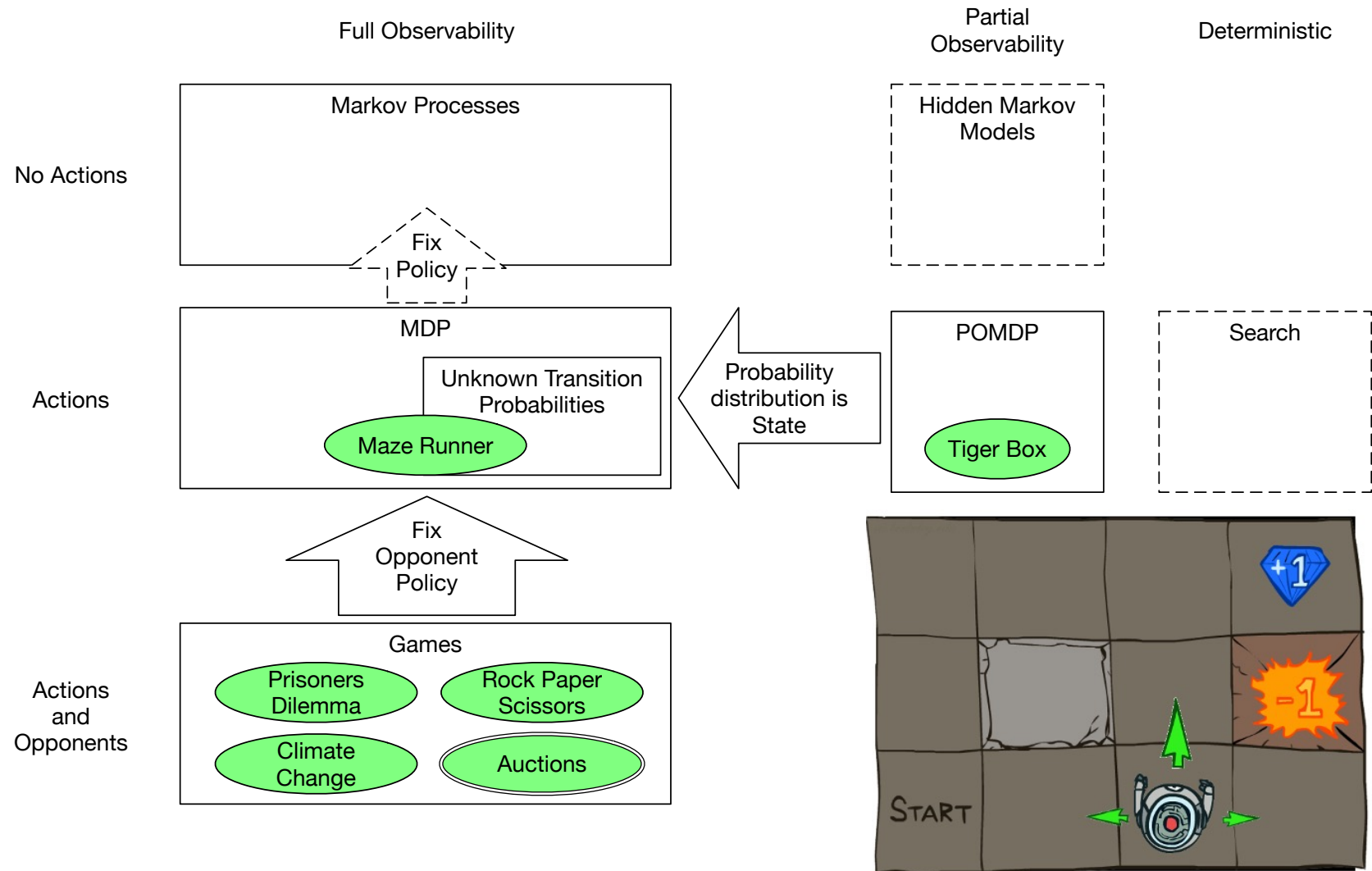
- Government procurement
- Online ads



Mechanism design

A good design benefits both the individual and the system!
Can be applied to many areas of society as well

Problem Classes





Credits

Original slides:

- Iolanda Leite, KTH
- Patric Jensfelt, KTH

Based partly on materials from:

- Berkeley AI
- Duke University AI (CPS 270)



Thank you!